

Python Programming

Semester III — Detailed Unit-wise Notes (Course Code: 2418305)

Unit 1.0 — Fundamentals of Python Programming Syntax

1.1 Introduction to Python — Character Set, Tokens, Variables, L-value/R-value, Comments

Python is a high-level, interpreted, general-purpose programming language known for readability and concise syntax.

- Character Set:** the set of valid characters Python recognizes — letters (A-Z, a-z), digits (0-9), special symbols, and whitespace.
- Tokens:** the smallest individual units of a program — keywords, identifiers, literals, operators, and punctuators.
- Variables:** named locations in memory used to store data; Python variables do not need explicit type declaration (dynamically typed) — a name is bound to an object when assigned.
- L-value and R-value:** an **L-value** refers to a memory location (something that can appear on the left of an assignment, e.g., a variable name); an **R-value** refers to the value/expression on the right side of an assignment that gets stored.
- Comments:** non-executable text used to explain code, written using `#` for single-line comments or triple quotes for multi-line documentation strings.

1.2 Overview of Data Types

- Number Types:**
 - Integer (int)** — whole numbers, e.g., 10, -5.
 - Floating Point (float)** — decimal numbers, e.g., 3.14.
 - Complex** — numbers with a real and imaginary part, e.g., 3+4j.
 - Boolean Type (bool):** represents True or False.
- Sequence Types:**
 - String (str)** — an ordered, immutable sequence of characters.
 - List** — an ordered, **mutable** collection of items, e.g., [1, 2, 3].
 - Tuple** — an ordered, **immutable** collection of items, e.g., (1, 2, 3).
 - None Type:** represents the absence of a value (NoneType, value None).
- Mapping Type: Dictionary (dict)** — an unordered (insertion-ordered since Python 3.7) collection of key-value pairs, e.g., {"a": 1}.
- Mutable vs Immutable:** mutable types (list, dict, set) can be changed after creation; immutable types (int, float, str, tuple) cannot be changed once created — any "modification" creates a new object.

1.3 Understanding Operators

- Arithmetic Operators:** +, -, *, / (float division), // (floor division), % (modulus), ** (exponent).
- Relational Operators:** ==, !=, >, <, >=, <= — return Boolean results.
- Logical Operators:** and, or, not — combine Boolean expressions.

bullet **Assignment Operator:** = assigns a value to a variable.

bullet **Augmented Assignment Operators:** +=, -=, *=, /=, etc., combine an operation with assignment, e.g.,
x += 1 is equivalent to x = x + 1.

bullet **Expressions and Statements:** an **expression** evaluates to a value (e.g., 2 + 3); a **statement** is a complete instruction (e.g., x = 2 + 3).

bullet **Type Conversion:**

bullet **Implicit** — Python automatically converts a smaller type to a larger one (e.g., int to float during an operation).

bullet **Explicit** — using functions like int(), float(), str() to manually convert types.

bullet **Input/Output:** input() reads a string from the user; print() displays output.

bullet **Precedence of Operators:** defines the order in which operators are evaluated in an expression (e.g., exponentiation is evaluated before multiplication/division, which are evaluated before addition/subtraction).

bullet **Expression Evaluation:** follows the operator precedence rules along with associativity (left-to-right or right-to-left) to determine the final computed result.

Example

a = 10

b = 3

print(a // b, a % b, a ** b) # 3 3 1000

Unit 2.0 — Conditional and Iterative Statements

2.1 Conditional Statements

Simple if statement: executes a block only if the condition is True.

```
if a > b:
    print("a is greater")
```

if-else statement: executes one block if the condition is True, another if False.

if-elif-else statement: tests multiple conditions in sequence, executing the block for the first True condition; the final `else` runs if none match.

2.2 Iterative Statements

while loop: repeats a block as long as a condition remains True — used when the number of iterations is not known in advance.

for loop: iterates over a sequence (list, string, tuple, range, etc.), executing the block once per element.

range() function: generates a sequence of numbers, commonly used to control the number of iterations in a for loop, e.g., `range(0, 10, 2)`.

break statement: immediately terminates the nearest enclosing loop.

continue statement: skips the rest of the current iteration and moves to the next.

Nested loops: a loop placed inside another loop, used for multi-dimensional iteration (e.g., processing rows and columns of a matrix).

```
for i in range(1, 4):
    for j in range(1, 4):
        if j == 2:
            continue
        print(i, j)
```

Unit 3.0 — String, List, Tuples, Set and Dictionary

3.1 String

- bullet **Indexing:** each character of a string can be accessed using a positional index, starting from 0 (or -1 from the end).
- bullet **String operations:**
- bullet **Concatenation** (+) — joins two strings.
- bullet **Repetition** (*) — repeats a string n times.
- bullet **Membership** (in, not in) — checks if a substring exists.
- bullet **Slicing** (s[start:stop:step]) — extracts a portion of the string.
- bullet **Traversing a string using loops:** iterating character by character with a for loop.
- bullet **Built-in functions/methods:** len(), upper(), lower(), strip(), split(), replace(), find(), etc.

3.2 Lists

- bullet **Introduction:** a list is an ordered, mutable collection that can hold mixed data types.
- bullet **Indexing:** accessing elements by position, similar to strings.
- bullet **List operations:** concatenation (+), repetition (*), membership (in), slicing (list[a:b]), traversal using loops.
- bullet **Built-in list functions/methods:** append(), insert(), remove(), pop(), sort(), reverse(), len(), index(), count().
- bullet **Linear search on a list:** scanning each element sequentially to find a target value.
- bullet **Counting frequency of elements:** using a loop with a dictionary/counter, or the built-in count() method, to tally occurrences of each value.

3.3 Tuples

- bullet **Creating and initializing:** tuples are created using parentheses, e.g., t = (1, 2, 3), and are immutable once created.
- bullet **Accessing elements:** via indexing, similar to lists.
- bullet **Tuple assignment:** multiple variables can be assigned simultaneously from a tuple, e.g., a, b = 1, 2 (also enables easy swapping: a, b = b, a).
- bullet **Tuple methods and built-in functions:** count(), index(), len(), sorted().
- bullet **Nested tuples:** tuples that contain other tuples as elements, useful for representing structured/grouped data.

3.4 Set

- bullet **Creating a set:** an unordered collection of unique elements, created using {} or set().
- bullet **Traversing, adding, removing:** iterate with a for loop; add elements with add(); remove with remove()/discard().
- bullet **Set operations:**

bullet **Union (| or union())** — all elements from both sets.

bullet **Intersection (& or intersection())** — common elements.

bullet **Difference (- or difference())** — elements in one set but not the other.

3.5 Dictionary

bullet **Accessing items using keys:** `d[key]` retrieves the value associated with a key.

bullet **Mutability:**

bullet **Adding a new item:** `d["new_key"] = value`.

bullet **Modifying an existing item:** assigning a new value to an existing key.

bullet **Built-in dictionary functions/methods:** `keys()`, `values()`, `items()`, `get()`, `update()`, `pop()`, `len()`.

Unit 4.0 — Python Functions, Modules and Packages

4.1 Functions

A **function** is a reusable, named block of code that performs a specific task.

bullet **Types of functions:**

bullet **Built-in functions** — provided by Python itself (e.g., `len()`, `print()`).

bullet **Functions defined in a module** — imported from Python's standard library or third-party packages.

bullet **User-defined functions** — created by the programmer using the `def` keyword.

bullet **Creating a user-defined function:**

```
def greet(name):  
    return f"Hello, {name}"
```

bullet **Arguments and parameters:** **parameters** are variables listed in the function definition; **arguments** are the actual values passed when calling the function.

bullet **Default parameters:** parameters with a default value used if no argument is supplied, e.g., `def greet(name="World")`.

bullet **Positional parameters:** arguments matched to parameters based strictly on their order.

bullet **Lambda functions:** small, anonymous, single-expression functions defined with the `lambda` keyword, e.g., `square = lambda x: x*x`.

bullet **Returning a value:** the `return` statement sends a result back to the caller; a function without `return` implicitly returns `None`.

bullet **Scope of a variable:**

bullet **Global scope** — variables defined outside any function, accessible throughout the program.

bullet **Local scope** — variables defined inside a function, accessible only within that function.

4.2 Modules and Packages

bullet **Modules:** a `.py` file containing Python code (functions, classes, variables) that can be reused by importing it with the `import` statement.

bullet **Regular Expressions (re module):** used for pattern matching and text searching/manipulation using special syntax (e.g., `re.match()`, `re.search()`, `re.findall()`).

bullet **Exception Handling:** mechanism to handle runtime errors gracefully (detailed further in Unit 6).

bullet **PyPI (Python Package Index):** the official online repository of third-party Python packages.

bullet **Pip:** the standard package manager used to install/manage packages from PyPI, e.g., `pip install numpy`.

bullet **Importing Libraries and Functions:** using `import module_name` or `from module_name import function_name` to bring external code into a program.

Unit 5.0 — Numpy

5.1 Introduction to NumPy

NumPy (Numerical Python) is a powerful library for numerical and scientific computing in Python, providing support for large, multi-dimensional arrays and a wide range of mathematical functions to operate on them efficiently.

5.2 Installation of NumPy

NumPy is installed using the package manager: `pip install numpy`, and imported using `import numpy as np`.

5.3 NumPy Arrays

- The fundamental data structure:** the `ndarray` (n-dimensional array) — unlike Python lists, all elements must be of the same data type, enabling fast, vectorized computation.
- Creation of arrays:** `np.array([1, 2, 3])`, `np.zeros((2, 3))` (array of zeros), `np.ones((2, 3))` (array of ones), `np.arange()`, `np.linspace()`.
- Array attributes:** `.shape` (dimensions), `.size` (total number of elements), `.ndim` (number of dimensions/axes).

5.4 Array Indexing and Slicing

- Indexing/slicing:** accessing individual elements or sub-arrays using `array[i]`, `array[i:j]`, similar to lists but extended to multiple dimensions.
- One-dimensional vs multi-dimensional indexing:** a 1-D array uses a single index `arr[i]`; a 2-D array requires two indices `arr[i][j]` or `arr[i, j]` to access a specific row and column.

5.5 Array Operations

- Element-wise operations:** arithmetic operators (+, -, *, /) applied directly between arrays operate element by element (vectorized, much faster than explicit loops).
- Universal Functions (ufuncs):** NumPy functions (e.g., `np.sqrt()`, `np.exp()`, `np.sin()`) that operate element-wise on arrays efficiently.

5.6 Linear Algebra with NumPy

NumPy's `linalg` module provides linear algebra operations:

- Matrix multiplication:** using `np.dot()` or the `@` operator.
- Determinant:** `np.linalg.det()`.
- Inverse:** `np.linalg.inv()`.
- Solving linear equations:** `np.linalg.solve(A, b)` solves the system $Ax = b$.

5.7 File Input and Output with NumPy

NumPy provides functions such as `np.save()/np.load()` (binary `.npy` format) and `np.savetxt()/np.loadtxt()` (text format) to write and read array data to/from files.

5.8 Broadcasting in NumPy

Broadcasting allows NumPy to perform arithmetic operations on arrays of different (but compatible) shapes by automatically "stretching" the smaller array across the larger one, without actually copying data, making operations like adding a scalar to a matrix efficient and convenient.

Unit 6.0 — Exception and File Handling in Python

6.1 Exception Handling

- bullet **Syntax errors:** errors in the code's grammar, detected before execution (e.g., missing colon).
- bullet **Exceptions:** errors detected during execution that disrupt the normal flow of a program (e.g., dividing by zero).
- bullet **Need for exception handling:** prevents the program from crashing abruptly and allows graceful recovery or informative error reporting.
- bullet **User-defined exceptions:** custom exception classes created by inheriting from the built-in `Exception` class.
- bullet **Raising exceptions:** using the `raise` statement to deliberately trigger an exception.
- bullet **Handling and catching exceptions:**

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print("Cannot divide by zero:", e)
else:
    print("Result:", result)
finally:
    print("Execution completed")
```
- bullet **Try-except-else clause:** the `else` block runs only if no exception occurred in the `try` block.
- bullet **Try-finally clause:** the `finally` block always executes, regardless of whether an exception occurred — used for cleanup actions (closing files, releasing resources).
- bullet **Built-in exception classes:** `TypeError`, `NameError`, `IndexError`, `KeyError`, `ValueError`, `IOError`, `ZeroDivisionError`, etc., each representing a specific category of error.

6.2 File Handling

- bullet **Text file vs Binary file:** text files store human-readable character data; binary files store data in raw byte format (e.g., images).
- bullet **File types:** determined by content and extension (.txt, .csv, .bin, etc.).
- bullet **Opening and closing files:** `open(filename, mode)` opens a file and returns a file object; `close()` releases the file resource (or use a `with` block for automatic closing).
- bullet **Reading and writing text files:** methods such as `read()`, `readline()`, `readlines()` for reading; `write()`, `writelines()` for writing.
- bullet **Reading and writing binary files:** performed using modes like `"rb"` and `"wb"`, handling raw bytes instead of text.
- bullet **File access modes:**
 - bullet `"r"` — read (default).
 - bullet `"w"` — write (overwrites existing content).
 - bullet `"a"` — append (adds to the end of the file).
 - bullet `"r+", "w+", "a+"` — combined read/write modes.
 - bullet `"b"` suffix (e.g., `"rb"`, `"wb"`) — binary mode.